Esha Tripathi z5391046
Venus Baranwal z5483461
Rukhsaar Aujla z5619382
Netik Kumar Maheshwari z5636903

COMP4601

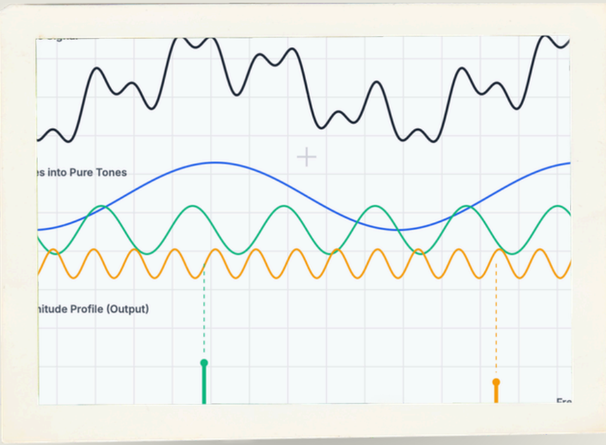
Audio Fingerprinting Acceleration

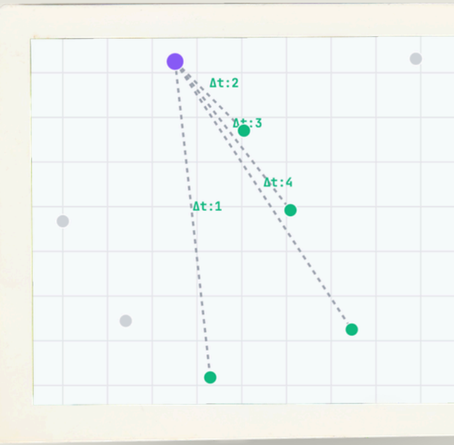
Project Overview

Audio fingerprinting generates compact signatures to quickly identify songs. The algorithm we're investigating is a C++ port of Dejavu, a popular audio fingerprinting algorithm originally written in Python.

AIM: Profiling a C++ port of the Dejavu algorithm revealed that 2D local peak detection dominated runtime, accounting for 97.5% of execution time (16.28s). We aim to accelerate this hotspot using HLS pragmas and evaluate the performance gains.

RESULT: We achieved a **4,071x** kernel speedup (16.28s to 4ms!) and a **16.5x** end-to-end speedup (16.71s to 1.01s).





Windowing (Slice + Hann)

- 22.05 kHz PCM input
- 4096-sample windows
- 50% overlap (HOP = 2048)
- Hann windowing

FFT + Spectrogram

- Compute 1D FFT per frame
- Log-power magnitude
- Map to 2D array
- Freq vs. Time grid

Peak Detection

- 41x41 Diamond mask
- Find local maxima
- Threshold filter (A > 10)
- Output (freq, time)

Hashing

- Anchor peak targeting (Fan-out = 15)
- Key: [f1 | f2 | Δt] & SHA-1 digest
- Store [Hash, Time_Offset] in JSON
- Histogram alignment & time match

Design Methodology (Simplify, Verify, Investigate, Accelerate)

Preparing the software algorithm for hardware synthesis required significant restructuring.

SOFTWARE DESIGN

- We **rewrote the code** to remove dependencies like opencv and boost and used standardised HLS friendly data structures like arrays and structs instead of vectors.
- We **iteratively improved** the code (v2, v3, v4) to ensure synthesis compatibility and optimal memory access before we attempted HLS acceleration .
- We **verified** our code using a **test bench** after each iteration to ensure functional correctness against the source code.

HARDWARE DESIGN

- Once satisfied with our SW simplifications, we **ran our code** on the ARM processor (PS) and **profiled** each function using the **chrono** library. We also profiled the transfer overheads.
- The profiling results showed that peak detection accounted for **97.45%** of the program's total runtime (16284.9 ms out of 16699.8ms total).
- We used the **synthesis** results to **find** which **loops** dominated runtime and **looked for acceleration** opportunities (unrolling, pipelining etc).
- We **compared** different iterations by their **latency, throughput, execution time, speedup and resource utilisation**.
- Rinse, repeat!

AI Assistance: AI tools(Gemini) helped in debugging vitis linking/timing errors and simplifying the code to remove original dependencies. It was also used to generate the script for the demo (big mistake). AI was (intentionally) not used in any other capacity because over-reliance ruins the learning experience and reduces the quality of output produced.

SW Profiling Early Stages

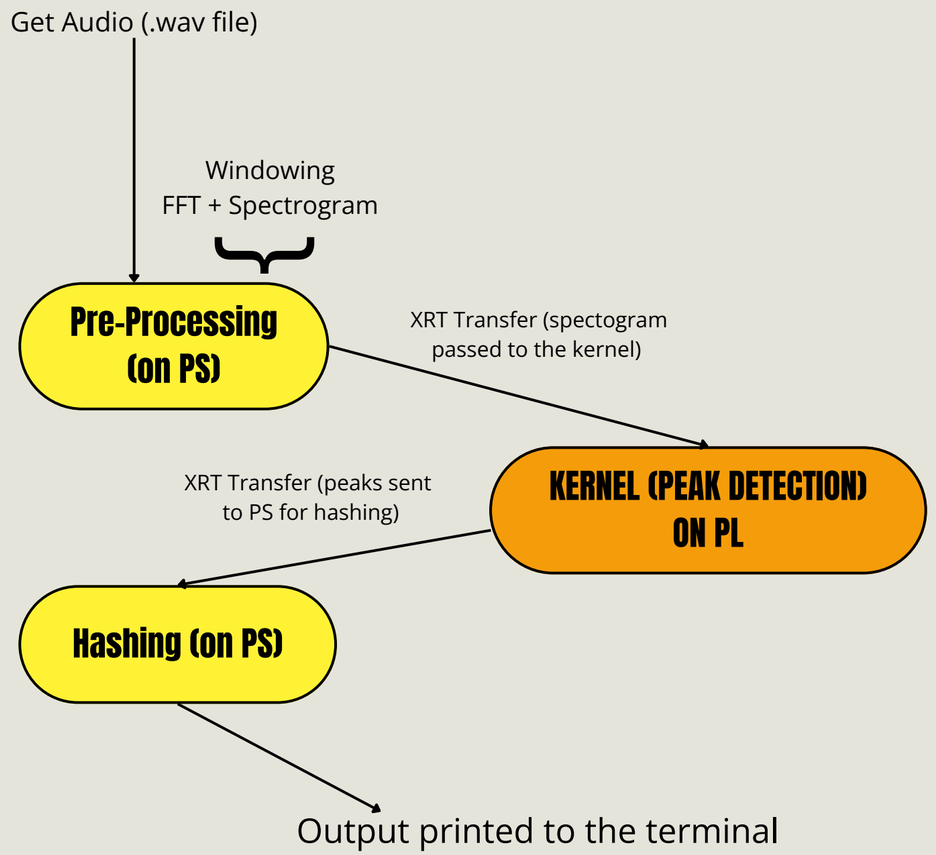
HW-SW Interface

MacBook (Early Profiling in Week 3)

Stage	Time (ms)
Windowing	0.669709
FFT+Spectrogram	18.6549
Peak Detection	**1024.96**
Hashing and JSON	7.99
Total	1,052.27

SW-Only on KRIA KV260

Stage	Time (ms)
Windowing	14.86
FFT+Spectrogram	392.447
Peak Detection	**16284.7**
Hashing and JSON	7.855
Total	16692 ms



Since Peak Detection dominates runtime (97.5%), we decided to move into a kernel for acceleration. The HW-SW Interface shows the separation of stages across the PS and PL.

HW DESIGN ITERATIONS

Version	Optimisation	LUT (%)	FF (%)	BRAM (%)	II / Latency Issue	Total Latency (ms)	Speedup Factor (x)	Throughput (Windows/s)
V1 (Unaccelerated)	Baseline HLS code; II=3 memory dependency on find_max.	6.28% (7.3k)	1.83% (4.2k)	7.29%	Latency: 25.3 µs per loop	5,005.00	1x	0.2
V2 (Full Unroll)	Naive #pragma HLS UNROLLgenerated 841 raw comparators without tree reduction.	169.15%	138.02%	14.93%	Over-utilised logic	21.287	235.1x	46.98
V3 (Bit-width Reduction)	Switched to ap_fixed<32,16> & cyclic partition; memory bottleneck persisted.	85.00% (100k)	94.00% (221k)	5.00%	II Memory Dependency	50	100.1x	20
V4 (Partial unrolling)	Added partial partitioning and unrolling instead to save resources but still exceedded the limits	171% (200k)	123.00% (288k)	15.00%	Over-budget	18.5	270.5x	54.05
V5 (Static Mask)	Added compile-time DiamondMask; pruned invalid gates but still exceeded logic.	149.00% (174k)	105.00% (245k)	14.00%	Over-budget	14.35	348.8x	69.69
V6 (AXI Burst Fix)	Fixed AXI burst transfers; relaxed II to fit within resource limits.	79.00% (92.5k)	60.00% (140k)	24.00%	Fits device (19.9 ms)	19.9	251.5x	50.25
V7 (Golden Version)	Balanced 1024-node Tree Reduction + Local BRAM peak staging.	**87.00% (101k)**	**67.00% (156k)**	**24.00%**	**Pipelined II=1 (4.00 ms)**	**3.514**	**1424.3x**	**284.58**

PSEUDO CODE: OPPORTUNITIES FOR ACCELERATION

```
// Main outer loop flushes spectrogram through line buffer
slide_window: for (int f = 0; f < MAX_FREQ + R; f++) {
    read_row: burst_read_DDR_to_local_row(new_row); // AXI Burst Read

    shift_window: for (int w = 0; w < num_windows + R; w++) {
        #pragma HLS PIPELINE II=1

        // 1. Shift 41x41 register window left by 1 column
        //    (ideally in 1 clock cycle)
        shift_register_grid_left();

        // 2. Feed new BRAM line-buffer column into right edge of window
        load_new_column_from_line_buf();

        // 3. Parallel Balanced Tree Reduction over 841 points
        DTYPE max_val = find_diamond_max(window);

        // 4. Store peak in local BRAM to prevent AXI stall
        if (window[20][20] == max_val && window[20][20] > AMP_MIN) {
            local_peak_buf[pc++] = {f_center, w_center};
        }
    }
}
write_axi: burst_write_local_peaks_to_DDR(); // Sequential AXI Burst Output
```

LINE BUFFERS

Reading 41 separate rows directly from main memory (DDR) forces 41 slow, serial lookups. Caching 40 previous rows in BRAM lets us fetch a full 41-pixel vertical column every cycle while reading only 1 new row from DDR.

WINDOW SHIFT

BRAM's 2-read limit forces an 840-cycle stall loop.
So, we can use flip-flops here!
Mapping 1,681 elements to registers reduces shift latency

BINARY TREE MAX REDUCTION

840 sequential comparisons violate clock timing and II=1. We can use an inline 10-level parallel tree $\log_2(841)=10$ evaluates 841 points in 1 cycle.

PEAK STAGING AND BURST AXI WRITE

Conditional in-loop writes introduce non-deterministic bus stalls, breaking pipeline flow. Instead, we can store detected peaks in local BRAM first, then burst-write the entire batch to DDR after the loop finished.

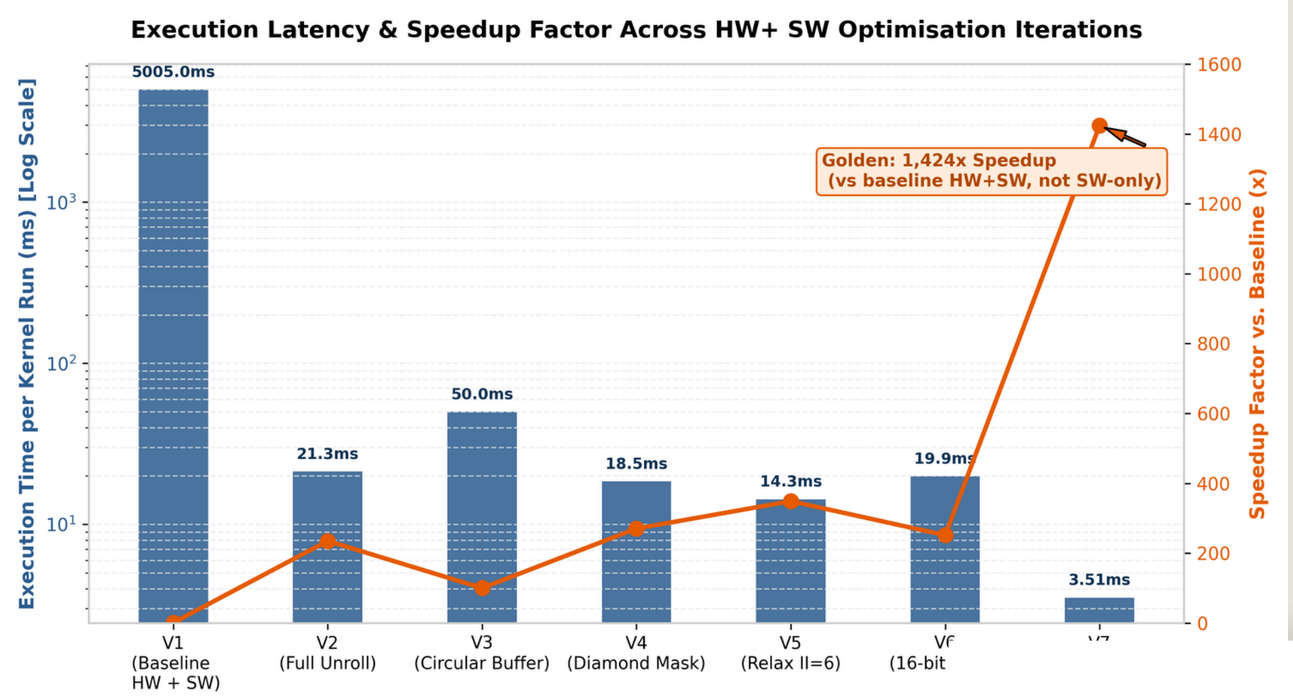


Figure 1: Execution time (log scale) and speedup plotted for each iterative optimisation. The baseline implementation (V1) required 5,005 ms per kernel run. Early optimisation attempts (V2–V6) dropped latency below 22 ms but hit memory dependency bottlenecks (II>1) and resource limits. The final architecture (V7) resolved these dependencies via a static diamond mask and balanced tree reduction, reaching a 3.51 ms latency i.e. a 1,424x speedup over the baseline.

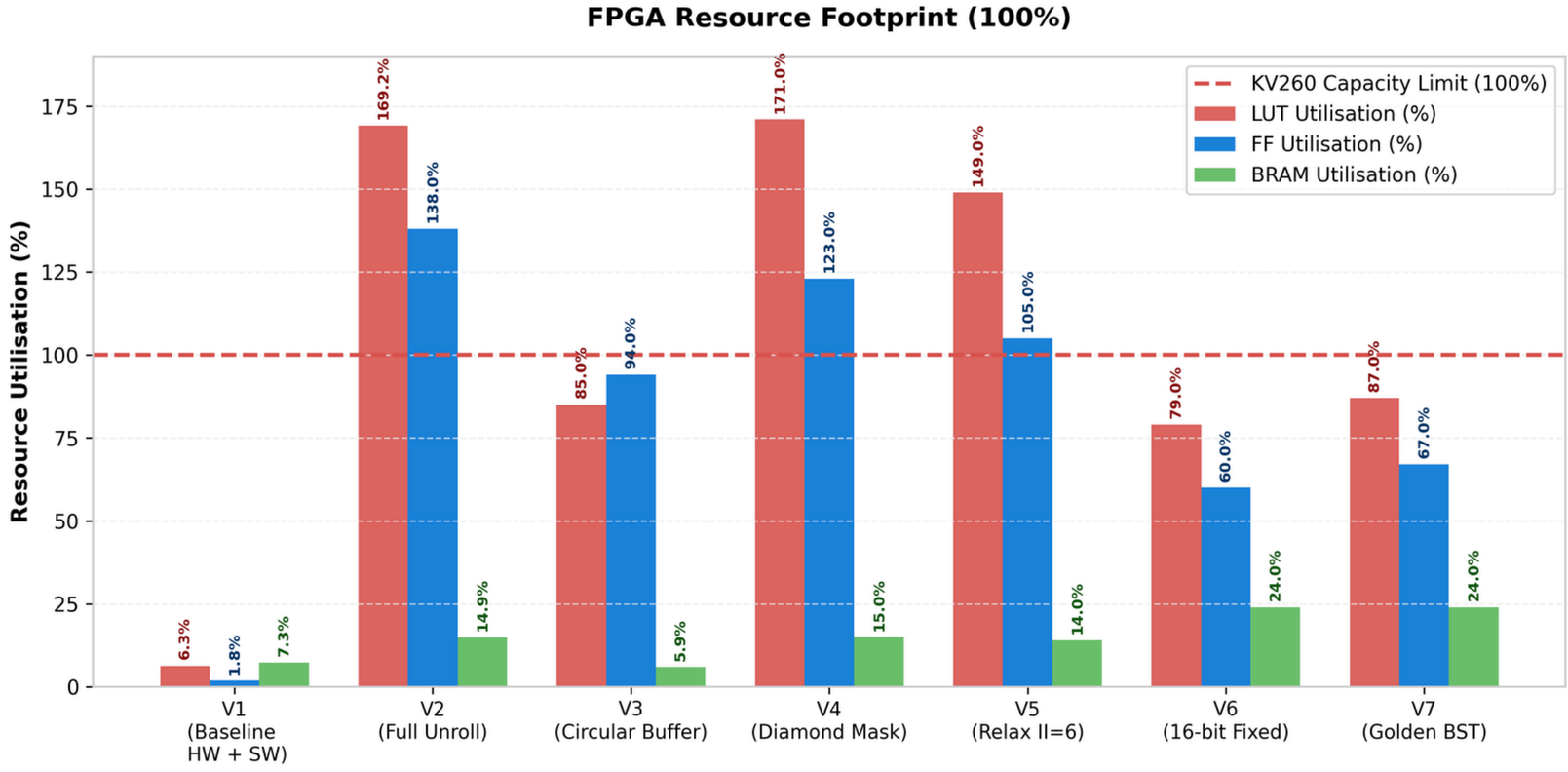
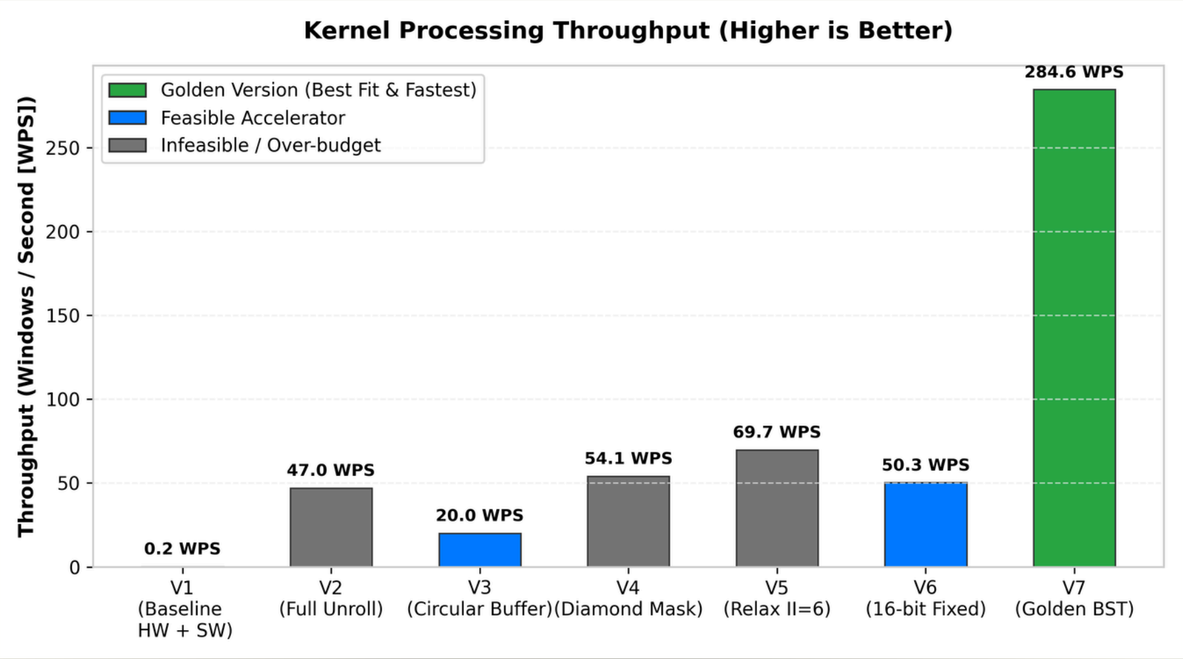


Figure 2: Resource footprint per iteration. Naive loop unrolling and partitioning (V2, V4, V5) over-allocated logic, peaking at 171% LUT usage (infeasible). Incorporating 16-bit fixed-point representation (ap_fixed<16,10>), uint16s and static mask routing in V6 brought the design within budget and using BST to find max gave 87% LUT, 67% FF, 24% BRAM (>V6 but better latency).

Results



4,071x Peak Detection Speedup

Peak Detection Kernel Execution Comparison:

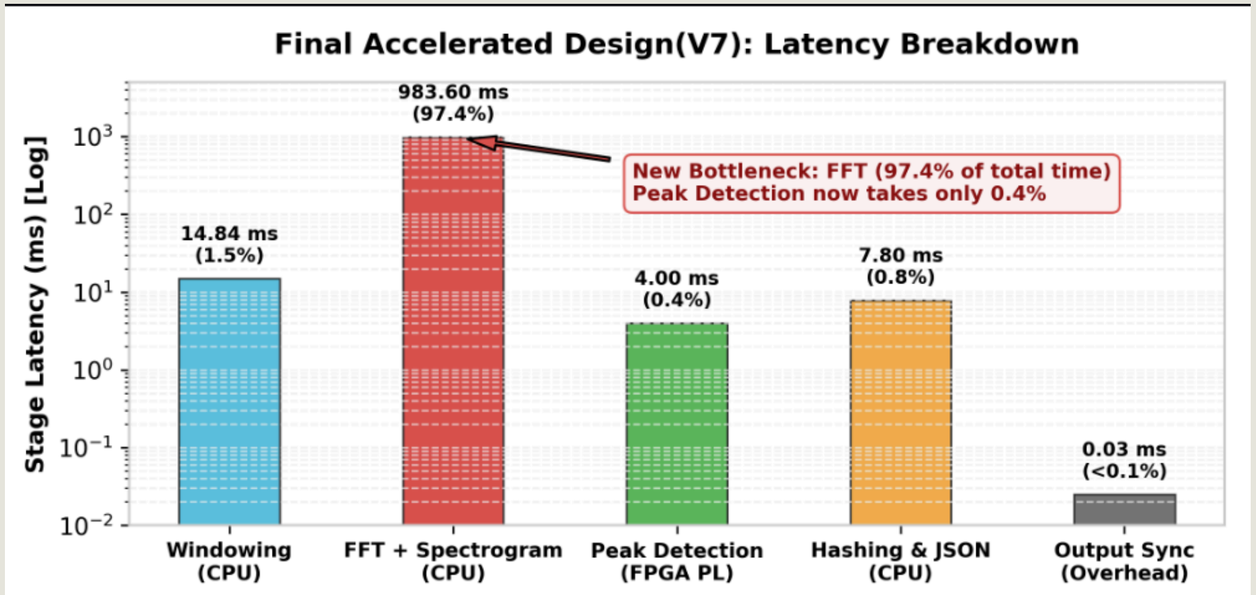
- **S**W Only (ARM PS): 16,284.90 ms
- SW + HW V1 (Unaccelerated): 5,005.44 ms (3.25x speedup)
- SW + HW V6 (Accelerated Option 1): 16.51 ms (986x speedup)
- SW + HW V7 (Golden Version Option 2): 4.00 ms: 4,071x Speedup!

Full System End-to-End Timing Breakdown (Golden V7):

- Windowing (CPU): 14.84 ms
- FFT + Spectrogram (CPU): 983.60 ms
- Peak Detection (FPGA PL + XRT Transfer): 4.00 ms
- Hashing & JSON (CPU): 7.80 ms
- Output Sync Overhead: 0.02 ms

Total Pipeline Execution: ~1,010.26 ms (Down from 16.71 seconds)

Lessons learned and Future Work



WHAT DIDN'T WORK

- Unconstrained Array Partitioning is expensive: Fully unrolling nested loops over a 41x41 grid without reduction trees blew up LUT utilisation by 169%.
- Conditional Memory Writes inside Pipelined Loops is bad: Direct AXI memory writes inside conditionally executed loops broke AXI burst generation. Solved by staging results in local BRAM buffers before burst-flushing.

FUTURE WORK

- **C**urrently FFT + Spectrogram is 983.60 ms of ~1010 ms total (~97.4 % of runtime), running at 172 FFTs/sec on the host
- In the future, FFT and peak detection could be moved into a single HLS kernel so the spectrogram streams directly on-chip between stages, removing the PS↔PL round-trip currently bundled into the peak-detection timing
- Fused kernel/host interface: host sends raw windowed audio in → single kernel does FFT → spectrogram → peak detection on-chip → only peak coordinates returned - cutting transfer overhead to one round-trip instead of one per stage
- We should have a CPU_SPECIFIC bitwidth to avoid unnecessary type conversions (using floats instead of ap_fixed) during our preprocessing.